Department of Information Technology

ELECTRICITY BILL CALCULATOR

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

Submitted By

Team Members

SHAPTHAGIRI G	**814725205105**
RENGESH M	**814725205088**
SAI SARAN N	**814725205066**

Under the Guidance of

Mrs. C. Manonmani

SRM TRP ENGINEERING COLLEGE

(An Autonomous Institution Affiliated to Anna University, Chennai)

Tiruchirappalli – 621 105

Academic Year: 2026-27 (ODD)

SRM TRP ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY
TIRUCHIRAPPALLI

AUTONOMOUS

Department of Information Technology

BONAFIDE CERTIFICATE

This is to certify that **SHAPTHAGIRI G (Reg. No: 814725205105), RENGESH M (Reg. No: 814725205088), and SAI SARAN N (Reg. No: 814725205096)** from second year B.Tech Information Technology have successfully completed their mini-project work titled "**ELECTRICITY BILL CALCULATOR**" at SRM TRP Engineering College, Tiruchirappalli in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology.

Mrs.C.Manonmani
(Faculty In-charge)
Artificial Intelligence &
Data Science

Dr. S. Sabeetha Saraswathi
(Head of Department)
Information
Technology

ACKNOWLEDGEMENT

It is with sincere gratitude and deep appreciation that we take this opportunity to acknowledge the invaluable support, constructive advice, and continuous guidance we have received throughout the course of this mini-project work.

We express our profound heartfelt thanks to our respected Project Guide, Mrs. C. Manonmani, Faculty In-charge, Department of Artificial Intelligence & Data Science, for her scholarly guidance, constant encouragement, and insightful feedback during every stage of designing and developing the Electricity Bill Calculator application.

We extend our heartfelt gratitude to Dr. [Head of Department], Head of the Department of Information Technology, for providing the necessary infrastructural resources, computer laboratory facilities, and an encouraging academic environment that fostered technological innovation and experiential learning.

We would also like to record our deep sense of gratitude to the Principal, Executive Director, and the Management of SRM TRP Engineering College, Tiruchirappalli, for providing state-of-the-art computational facilities and continuous academic support.

A special note of thanks goes to all the faculty members, laboratory instructors, and supporting technical staff of the Department of Information Technology for their kind cooperation, technical assistance, and timely guidance throughout the project tenure.

Finally, we express our deepest, warmest gratitude to our beloved parents and family members for their unconditional love, moral encouragement, patient endurance, and constant inspiration. We also thank our fellow classmates and friends who directly or indirectly contributed their valuable suggestions towards the successful completion of this project work.

Academic Year: 2026-2027

Team Members:

SHAPTHAGIRI G (814725205105)

RENGESH M (814725205088)

SAI SARAN N (814725205096)

TABLE OF CONTENTS

S.No.	Title / Section	Page No.
1	Acknowledgement	3
2	Abstract	6
3	List of Tables	7
4	List of Figures	8
5	Chapter 1: Introduction	9
	1.1 Project Background	9
	1.2 Problem Statement	10
	1.3 Aim of the Project	10
	1.4 Objectives of the Project	10
	1.5 Scope of the Project	11
	1.6 Need for the Project	11
	1.7 Existing System and Drawbacks	12
	1.8 Proposed System and Advantages	12
	1.9 Organization of the Report	13
6	Chapter 2: System Requirements	14
	2.1 Hardware Requirements	14
	2.2 Software Requirements	14
	2.3 Functional Requirements	15
	2.4 Non-Functional Requirements	15
	2.5 Feasibility Study	16

S.No.	Title / Section	Page No.
	2.6 Technologies Used	17
7	Chapter 3: System Design	19
	3.1 Overall System Architecture	19
	3.2 Page Structure and Modules	20
	3.3 Consumer and Billing Data Design	21
	3.4 Progressive Tariff Slabs Specification	22
	3.5 Use Case Diagram and Analysis	23
	3.6 Data Flow Diagrams (DFD Level 0 & Level 1)	24
	3.7 Overall System Workflow	26
	3.8 Sequence Diagram	27
	3.9 Site Map and Navigation Structure	28
	3.10 User Interface Design & Wireframe	29
	3.11 Hosting and Deployment Design	30
8	Chapter 4: System Implementation	31
	4.1 Introduction to Implementation	31
	4.2 Frontend Structure Implementation (HTML5)	31
	4.3 Styling & Layout Implementation (CSS3)	32
	4.4 Client-Side Interactivity (JavaScript)	33
	4.5 Functional Modules Breakdown	34
	4.6 Mathematical Formulations & Slab Algorithm	35
	4.7 Input Validation and Boundary Handling	38

S.No.	Title / Section	Page No.
	4.8 Browser LocalStorage Persistence	39
	4.9 Responsive Design & Print Invoice Styling	40
	4.10 Security and Validation Considerations	41
	4.11 Output Screen Captures	42
	4.12 Source Code Screenshots	46
9	Chapter 5: Testing and Results	48
	5.1 Introduction to Software Testing	48
	5.2 Testing Objectives	48
	5.3 Types of Testing	49
	5.4 Test Environment	50
	5.5 Test Cases and Execution Results	51
	5.6 Testing Summary and Findings	54
10	Chapter 6: Result and Conclusion	55
	6.1 Results Achieved	55
	6.2 Key Advantages of the System	55
	6.3 System Limitations	56
	6.4 Future Enhancements	56
	6.5 Conclusion	57
11	References	58

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Technologies Used and Their Roles in the Project	17
Table 3.1	Sections and Functional Modules of the Web Portal	20
Table 3.2	Attributes of a Billing Record Data Model	21
Table 3.3	Illustrative Progressive Tariff Slabs Specification	22
Table 3.4	Use Case Specification and Actor Interactions	23
Table 5.1	Test Environment Configuration	50
Table 5.2	Functional Test Cases and Verification Results	51
Table 5.3	Interface and Cross-Browser Compatibility Test Cases	53
Table 5.4	Input Validation and Edge Case Test Suite	53

LIST OF FIGURES

Figure No.	Caption	Page No.
Figure 3.1	Overall Architecture of the Electricity Bill Calculator	19
Figure 3.2	Use Case Diagram of the Web Application	23
Figure 3.3	Data Flow Diagram – Level 0 (Context Level)	24
Figure 3.4	Data Flow Diagram – Level 1 (Functional Decomposition)	25
Figure 3.5	Bill Calculation and Application Workflow Flowchart	26
Figure 3.6	Sequence Diagram for Bill Calculation and History Storage	27
Figure 3.7	Site Map and Navigation Hierarchy of the Portal	28

Figure No.	Caption	Page No.
Figure 3.8	Wireframe Layout Plan of the Application Interface	29
Figure 4.1	Pseudo-code of the Progressive Slab Calculation Algorithm	37
Figure 4.2	Output Screen: Home Page and Hero Banner View	42
Figure 4.3	Output Screen: Illustrative Tariff Slabs Display Grid	43
Figure 4.4	Output Screen: Consumer & Meter Reading Input Form	43
Figure 4.5	Output Screen: Electricity Bill Summary and Slab Breakdown	44
Figure 4.6	Output Screen: Calculation History Table (LocalStorage)	44
Figure 4.7	Output Screen: Print-Optimized Invoice View	45
Figure 4.8	Output Screen: Input Validation Error Notification	45
Figure 4.9	Coding Screenshot: HTML Structure (index.html)	46
Figure 4.10	Coding Screenshot: JavaScript Logic (script.js)	47
Figure 4.11	Coding Screenshot: CSS Styling (style.css)	47

ABSTRACT

The Electricity Bill Calculator (branded as PowerCalc) is a lightweight, responsive, and client-side web application developed to simplify, automate, and streamline the computation of domestic and commercial electricity consumption and estimated utility charges. In conventional billing methods, consumers often experience ambiguity regarding how their electricity bill is derived across multi-tiered, progressive slab structures, frequently leading to misunderstandings, budgeting difficulties, and manual calculation errors. This project bridges that gap by offering a transparent, instant, and interactive web tool that enables power consumers to compute their energy consumption directly from meter readings and understand their progressive cost breakdown.

The web application provides an intuitive single-page interface comprising a prominent hero banner, an illustrative tariff slab overview, an interactive consumer and meter reading input form, a detailed progressive bill summary breakdown, persistent calculation history powered by browser LocalStorage, and a dedicated print stylesheet for generating clean invoice slips. Users input their previous meter reading and current meter reading in kilowatt-hours (kWh); the client-side JavaScript engine automatically computes the net energy units consumed (Units = Current Reading - Previous Reading) and rigorously applies a four-tier progressive tariff schedule (0–100 units at ₹1.50/unit, 101–200 units at ₹2.50/unit, 201–300 units at ₹4.00/unit, and units above 300 at ₹6.00/unit).

Developed using standard modern web technologies including HTML5 for semantic page layout, CSS3 for responsive and print styling, and modern JavaScript (ES6) for DOM manipulation and calculation logic, the system operates completely on the client side without requiring server-side runtimes or external database installations. Historical calculations are stored locally in the browser's LocalStorage key-value cache, permitting consumers to review past utility expenses or clear records at will. The project successfully demonstrates how foundational web engineering principles can be applied to solve everyday consumer utility challenges.

Keywords: Electricity Bill Calculator, Energy Consumption, Progressive Slab Tariff, Client-Side Web Application, LocalStorage, HTML5, CSS3, JavaScript, Responsive Design.

CHAPTER 1 INTRODUCTION

1.1 Project Background

Electricity is one of the most fundamental infrastructure resources powering modern civilization, supporting residential households, agricultural operations, commercial establishments, and heavy industrial facilities. Efficient energy utilization and transparent utility accounting are paramount for both power distribution companies and electricity consumers. Every billing cycle, power utility boards measure consumer consumption through electromechanical or digital energy meters located at customer premises. These meters record cumulative electrical energy consumed over time in kilowatt-hours (kWh), universally termed as "units".

To promote conservation, discourage wasteful energy consumption, and provide affordable baseline electricity to economically vulnerable citizens, power regulatory commissions across India adopt progressive, multi-tier slab tariff systems. Under a progressive slab scheme, energy charges do not follow a flat rate; rather, initial blocks of consumed units are priced at highly subsidized lower rates, whereas subsequent increments of consumption are assessed at progressively escalating marginal rates. While economically equitable, progressive slab tariffs are mathematically complex for ordinary citizens to calculate manually, causing confusion and dispute.

1.2 Problem Statement

Despite the widespread availability of digital energy meters, millions of consumers face persistent challenges in monitoring and verifying their electricity bills. In the conventional scenario:

- Consumers typically wait until the utility board issues a bi-monthly or monthly physical paper bill, leaving them unable to track their consumption trends proactively during the billing cycle.
- Manual calculation of progressive slab tariffs involves dividing total consumption across multiple rate tiers, which is highly error-prone and confusing for non-technical individuals.
- Consumers often mistakenly believe that crossing into a higher slab rate applies that higher tariff retroactively to all consumed units, leading to distrust in utility board accounting.
- There is a lack of simple, lightweight, zero-cost digital tools that allow householders and small business owners to enter previous and current meter readings, immediately verify consumption, inspect itemized slab costs, and maintain historical calculation records.

1.3 Aim of the Project

The primary aim of this mini-project is to design, develop, test, and deploy a responsive, highly accessible, and user-friendly web application titled "Electricity Bill Calculator" that

automatically calculates electricity consumption from meter readings and computes estimated progressive tariff charges using client-side web technologies.

1.4 Objectives of the Project

The specific academic and practical objectives established for this project include:

- To eliminate manual arithmetic errors by automating energy unit computation using the standardized formula (Units Consumed = Current Reading - Previous Reading).
- To implement an accurate progressive slab tariff calculation algorithm across four distinct consumption tiers (0-100 units @ ₹1.50, 101-200 units @ ₹2.50, 201-300 units @ ₹4.00, and above 300 units @ ₹6.00).
- To enforce strict client-side input validation preventing negative readings, blank fields, non-numeric inputs, and logical anomalies where the current reading is recorded as less than the previous reading.
- To generate a transparent, itemized electricity bill summary table clearly articulating the exact units billed and charges incurred within each individual tariff slab.
- To implement client-side data persistence using browser LocalStorage, enabling consumers to save, review, and delete previous calculation records locally without server overhead.
- To design a dedicated print stylesheet (@media print) allowing consumers to print an official, receipt-style electricity invoice summary directly from their browser.
- To build an aesthetically pleasing, responsive user interface supporting seamless interaction across desktop monitors, laptops, tablets, and smartphones.

1.5 Scope of the Project

The scope of this mini-project encompasses the complete frontend design and client-side behavioral engineering of the Electricity Bill Calculator application. Specifically, the scope includes:

- Interactive User Interface: Developing semantic HTML5 layouts with modern CSS3 styling, sticky navigation, visual tariff cards, responsive forms, and modular cards.
- Client-Side Computing: Implementing vanilla JavaScript (ES6) functions to process user inputs, perform real-time validations, execute tiered calculations, and update the Document Object Model (DOM) dynamically without page reloads.
- Data Persistence: Utilizing the browser Web Storage API (LocalStorage) to store JSON-formatted bill records persistently across browser sessions.
- Static Web Hosting: Designing the codebase as an independent, lightweight static site capable of direct hosting on platforms such as GitHub Pages.
- Academic Demonstration: Formulating illustrative domestic tariff slab schedules to demonstrate multi-tier utility calculation logic effectively.

1.6 Need for the Project

With the rapid adoption of digital governance and smart metering, consumer demand for

transparent self-service tools has increased exponentially. Enabling electricity consumers to calculate and audit their energy expenses fosters energy consciousness, assists in household financial planning, and prevents billing disputes. Because this application operates directly inside modern web browsers with zero installation prerequisites, it delivers universal accessibility across diverse devices and operating systems.

1.7 Existing System and Drawbacks

In the existing traditional system, meter readers visit premises periodically to note down readings manually or via handheld optical probes. Consumers receive printed billing receipts or SMS notifications after billing cycles close. The significant drawbacks of this approach are:

- No Real-Time Estimation: Consumers have no intuitive mechanism to calculate their mid-cycle consumption or forecast monthly utility costs before the official bill arrives.
- Opaque Calculation: Standard utility receipts often present aggregate energy charges without itemizing how units were segmented across progressive slabs, leaving consumers confused.
- Manual Calculation Errors: When consumers attempt to calculate bills manually using printed tariff charts, arithmetic errors in slab distribution frequently occur.
- Lack of Convenient Record Keeping: Paper receipts are easily misplaced, damaged, or lost, depriving householders of historical consumption records.

1.8 Proposed System and Advantages

The proposed Electricity Bill Calculator application addresses each of the aforementioned deficiencies by offering a clean, automated, and mathematically verified web interface. The key advantages of the proposed system are:

- Instant Automated Computation: Delivers immediate calculation results upon entering meter readings, saving time and eliminating manual arithmetic.
- Transparent Itemized Breakdown: Clearly displays slab-by-slab consumption units, marginal rates, and subtotal charges in a neat tabular breakdown.
- Zero Installation & High Portability: Operates on standard web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) across desktops, tablets, and smartphones.
- Client-Side Privacy & Offline Capability: Computations and historical records are processed locally via JavaScript and LocalStorage, requiring no external database or server transmission.
- Printable Digital Receipts: Incorporates customized print CSS formatting to produce professional, receipt-style invoices with a single click.

1.9 Organization of the Report

This mini-project report is organized into six comprehensive chapters structured as follows:

- Chapter 1 – Introduction: Presents project background, problem statement, aim,

objectives, scope, existing system limitations, proposed system benefits, and report outline.

- Chapter 2 – System Requirements: Outlines the hardware, software, functional, and non-functional requirements, technical and economic feasibility, and technical stack descriptions.
- Chapter 3 – System Design: Elaborates overall architecture, module structures, data models, tariff slab designs, use cases, DFD Level 0 & Level 1, workflow flowcharts, sequence diagrams, site maps, and UI wireframes.
- Chapter 4 – System Implementation: Details the technical realization of HTML5 structure, CSS3 styling, JavaScript algorithms, mathematical formulas, numerical examples, LocalStorage, responsive design, and output screenshots.
- Chapter 5 – Testing and Results: Documents testing methodologies, test environment configurations, test cases (functional, interface, validation), expected vs. actual outcomes, and verification summaries.
- Chapter 6 – Result and Conclusion: Discusses final project outcomes, achievements, limitations, potential future enhancements, and concluding remarks.

CHAPTER 2 SYSTEM REQUIREMENTS

System requirements specify the essential hardware resources, software platforms, development tools, and operational characteristics required to design, develop, execute, and maintain the Electricity Bill Calculator application. Because the application is architected as a lightweight, client-side static web application, the computational demands on both developer and client workstations are modest.

2.1 Hardware Requirements

The minimum and recommended hardware specifications required to run and execute the application are detailed below:

- Processor: Intel Core i3 or equivalent AMD multi-core processor (1.8 GHz or higher).
- RAM: Minimum 4 GB RAM (8 GB recommended for concurrent development and browser multitasking).
- Hard Disk Storage: Minimum 1 GB of free storage space for code files, documentation, and browser cache.
- Display Monitor: Standard color monitor with minimum resolution of 1366 × 768 pixels (the web layout is fully adaptive to mobile display resolutions down to 320px).
- Input Devices: Standard alphanumeric keyboard and optical mouse or touch-sensitive trackpad / touchscreen.
- Network Interface: Internet connection for initial resource downloading and remote repository hosting via GitHub.

2.2 Software Requirements

The software configuration and environment utilized during the project development lifecycle include:

- Operating System: Microsoft Windows 10 / Windows 11 (64-bit), Ubuntu Linux 20.04+, or Apple macOS.
- Frontend Technologies: HyperText Markup Language 5 (HTML5), Cascading Style Sheets 3 (CSS3), and JavaScript (ECMAScript 2020 / ES6+).
- Client Storage Mechanism: Browser Web Storage API (LocalStorage).
- Integrated Development Environment (IDE): Visual Studio Code (Version 1.85+) with Live Server extension.
- Version Control & Hosting: Git distributed version control system and GitHub Pages static web hosting service.
- Target Web Browsers: Google Chrome (v110+), Mozilla Firefox (v105+), Microsoft Edge (v110+), or Apple Safari (v15+).

2.3 Functional Requirements

Functional requirements describe the core operational capabilities, inputs, calculations, and

behaviors provided by the system to its end users:

- FR-1 (Consumer Profile Entry): The system shall accept consumer identification attributes including consumer name, service ID / consumer number, connection tariff type, and service address.
- FR-2 (Meter Reading Input): The system shall provide structured numeric input fields for recording previous energy meter reading and current energy meter reading in kWh.
- FR-3 (Strict Input Validation): The system shall validate that all required fields are filled, inputs contain valid non-negative numbers, and current reading is greater than or equal to previous reading.
- FR-4 (Consumption Calculation): The system shall automatically compute total energy consumption in units as the difference between current reading and previous reading.
- FR-5 (Progressive Slab Calculation): The system shall evaluate total consumption against a predefined four-tier progressive slab schedule, allocating units and calculating marginal costs for each applicable slab.
- FR-6 (Bill Summary Generation): The system shall dynamically generate an itemized invoice summary displaying consumer details, meter readings, units consumed, slab-wise breakdown, and net payable amount.
- FR-7 (LocalStorage History Management): The system shall allow users to persist calculated bill records into browser LocalStorage, display stored history records in a tabular list, and delete individual records or clear all history.
- FR-8 (Print Invoice Formatting): The system shall trigger browser printing with print-optimized CSS, hiding navigational headers, action buttons, and background colors to output a professional paper receipt.

2.4 Non-Functional Requirements

Non-functional requirements specify the operational quality attributes, performance benchmarks, and usability standards satisfied by the application:

- Usability & Simplicity: The user interface must be clean, highly intuitive, and self-explanatory, requiring zero technical training or prior instructions.
- Responsiveness: The application layout must dynamically adapt across standard screen viewports (desktop monitors, laptops, tablets, mobile smartphones) using CSS media queries and flexible grid layouts.
- High Performance & Low Latency: Because all calculations occur client-side via JavaScript without backend network round-trips, bill generation must execute instantaneously (under 50 milliseconds).
- Portability: The web application must operate uniformly across all major modern web browsers without requiring external browser plugins, compilers, or platform-specific libraries.
- Reliability & Error Prevention: The application must catch and handle invalid inputs gracefully, presenting clear visual error messages without application crashes or incorrect arithmetic outputs.
- Maintainability: Codebase must follow modular engineering practices, separating structure (HTML5), visual presentation (CSS3), and behavioral logic (JavaScript) into

decoupled files.

2.5 Feasibility Study

A feasibility study was conducted prior to implementation to analyze the technical, economic, and operational viability of developing the Electricity Bill Calculator.

2.5.1 Technical Feasibility

The project utilizes mature, industry-standard web standards: HTML5, CSS3, and JavaScript. These technologies are natively supported by every modern web browser on desktop and mobile platforms. No complex back-end application servers, web frameworks, or third-party database engines are mandatory for executing client-side mathematical calculations and storing local session data. Development tools such as Visual Studio Code provide excellent debugging and syntax verification. Thus, the project is technically sound and highly feasible.

2.5.2 Economic Feasibility

All software tools, compilers, development libraries, code editors (VS Code), web browsers (Google Chrome), and deployment environments (GitHub Pages) employed throughout this project are completely free, open-source, or zero-cost services. The application imposes zero recurring database licensing costs or dedicated cloud hosting infrastructure charges. Consequently, the project requires zero financial investment, making it economically optimal.

2.5.3 Operational Feasibility

The user interface is designed with familiarity and simplicity, mirroring standard online consumer utility portals. Householders, students, and small business owners can calculate bills effortlessly by simply inputting meter readings. Updating tariff slabs or modifying rate configurations requires only minor edits to a well-defined JavaScript configuration array. Hence, the project possesses high operational feasibility.

2.6 Technologies Used

Table 2.1 summarizes the technology stack selected for developing the Electricity Bill Calculator and the precise engineering role of each component.

Technology / Tool	Purpose and Engineering Role in Project
HTML5	Semantic structure: header, navigation, hero banner, tariff cards, calculator form, summary invoice, and footer.
CSS3	Visual design, color palette, flexbox and grid layouts, typography, button hover states, and responsive media queries.
JavaScript (ES6)	Client-side calculation engine, progressive slab logic, input validation, DOM event handling, and LocalStorage CRUD.
Web Storage API	Browser-native LocalStorage key-value persistence for storing,

Technology / Tool	Purpose and Engineering Role in Project
	retrieving, and clearing past calculation records.
Visual Studio Code	Modern source code editor utilized for authoring code, syntax highlighting, debugging, and project structuring.
Google Chrome / Edge	Primary runtime environments utilized for client execution, UI rendering, DevTools debugging, and invoice printing.
Git & GitHub Pages	Version control tracking, source repository management, and free static HTTPS web application deployment.

HTML5 provides semantic page organization using specialized structural tags such as <header>, <nav>, <section>, <form>, <table>, and <footer>. This semantic foundation ensures high accessibility and clean separation of concerns. CSS3 introduces sophisticated layout capabilities through CSS Flexbox and CSS Grid, supporting responsive breakpoints that reflow content seamlessly from wide monitors down to narrow smartphone screens. Modern JavaScript (ECMAScript 6+) runs directly in the client's browser sandbox, handling interactive events, validating user inputs, and computing progressive tariffs with zero server round-trip latency.

CHAPTER 3 SYSTEM DESIGN

3.1 Overall System Architecture

The Electricity Bill Calculator is architected following a modular client-side single-page architecture (SPA). Rather than relying on multi-tier server-side processing pipelines that require continuous client-server network exchanges, the entire application runtime executes inside the client web browser. When the user opens the application URL, the web server (or static host such as GitHub Pages) delivers the static HTML5, CSS3, and JavaScript files via HTTPS. Once loaded into browser memory, the presentation layer, business logic engine, and local storage manager operate entirely on the client workstation.

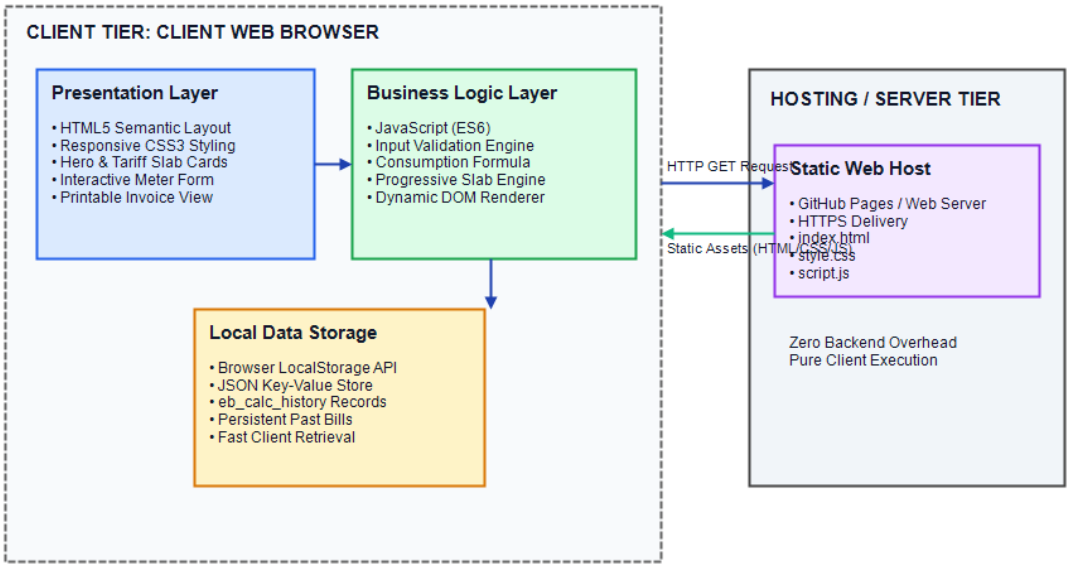


Figure 3.1: Architecture of the Electricity Bill Calculator

As illustrated in Figure 3.1, the client browser environment is divided into three coordinated subsystems:

- Presentation Subsystem (HTML5 & CSS3): Renders visual components including the navigation header, hero banner, interactive tariff slab grid, meter reading input form, calculation result card, and saved history tables.
- Business Logic Engine (JavaScript): Manages event listeners, sanitizes and validates consumer inputs, evaluates the mathematical difference between meter readings, executes the progressive slab calculation algorithm, and updates the Document Object Model (DOM).
- Data Persistence Subsystem (LocalStorage): Interacts with the browser's native Web Storage API to serialize calculation records into JSON strings, persist them under the key "eb_calc_history", and retrieve them for tabular display.

3.2 Page Structure and Modules

The application is organized into eight logical functional modules integrated seamlessly

within a cohesive single-page design. Table 3.1 delineates these modules and their operational purpose.

Module ID	Module Name	Functional Scope and Operational Purpose
Module 1	Header & Navigation	Sticky navigation bar featuring application branding, logo, and anchor links to Home, Tariffs, Calculator, History, and About.
Module 2	Hero Banner	Prominent introductory section communicating the application purpose, academic affiliation, and quick action buttons.
Module 3	Tariff Slabs Display	Interactive grid of four tariff cards displaying marginal rates, unit ranges, and terms for academic progressive demonstration.
Module 4	Consumer Input Form	Input interface capturing consumer name, service number, connection type, service address, previous reading, and current reading.
Module 5	Calculation Engine	Core JavaScript algorithms validating inputs, enforcing current >= previous, computing units, and calculating slab charges.
Module 6	Bill Invoice Summary	Dynamic result card displaying consumer profile, total units consumed, itemized slab breakdown table, and net payable bill.
Module 7	Calculation History	LocalStorage management module displaying past calculation records, supporting individual item deletion, and history clearing.
Module 8	Print Invoice Formatter	Print-optimized CSS module triggered by window.print() to generate clean, paper-ready invoice slips without navigation elements.

3.3 Consumer and Billing Data Design

To maintain data consistency and support structured persistence in LocalStorage, each bill calculation is modeled as a standardized JSON data entity. Table 3.2 details the attributes making up this data model.

Attribute Name	Data Type	Description and Representation Format
id	String	Unique calculation identifier generated using timestamp prefix (e.g., "BILL-1731168400000").
date	String	Formatted date and time of calculation formatted in Indian standard format (e.g., "09 Nov 2026, 09:30 PM").
consumerName	String	Full legal name of the electricity customer / consumer.
consumerNumber	String	Unique utility service connection identifier (e.g., "EB-2026-814725205105").
connectionType	String	Category of power connection (Domestic / Residential, Commercial, Small Industry).
consumerAddress	String	Physical premise address or geographical distribution area of the service point.
previousReading	Number	Initial cumulative energy meter reading recorded in kWh at billing cycle start.
currentReading	Number	Final cumulative energy meter reading recorded in kWh at billing cycle end.
unitsConsumed	Number	Net energy units consumed during the cycle (Current Reading - Previous Reading).
breakdown	Array[Object]	Itemized array containing slab label, units billed, marginal rate, and slab amount.
totalEnergyCharges	Number	Summation of progressive energy charges incurred across all applicable slabs.
fixedCharges	Number	Fixed meter rent / service charge (configured as ₹0.00 for academic demonstration).
grandTotal	Number	Net payable bill amount in Indian Rupees (₹) due from the electricity consumer.

3.4 Progressive Tariff Slabs Specification

The application implements a four-tier progressive domestic slab tariff schedule. Under this progressive structure, each portion of consumption is assessed exclusively at its respective slab rate. Table 3.3 presents the exact parameters configured in the calculation engine.

Slab Tier	Consumption Range	Slab Capacity	Marginal Rate	Intended Consumption Profile
Slab 1	0 to 100 Units	100 Units	₹1.50 per unit	Subsidized baseline consumption tier for essential household lighting and fans.
Slab 2	101 to 200 Units	100 Units	₹2.50 per unit	Moderate consumption tier covering basic electrical appliances and refrigeration.
Slab 3	201 to 300 Units	100 Units	₹4.00 per unit	Higher consumption tier covering air conditioning, water heaters, and washing machines.
Slab 4	Above 300 Units	Unlimited (Infinity)	₹6.00 per unit	Maximum marginal rate tier penalizing high energy consumption and encouraging efficiency.

** Important Clarification: The tariff rates tabulated above are illustrative parameters calibrated specifically for academic demonstration and laboratory testing of multi-tier progressive billing logic, and do not represent the statutory tariff orders issued by the Tamil Nadu Electricity Regulatory Commission (TNERC). Fixed charges, electricity duties, and state subsidies are excluded unless specifically enabled.*

3.5 Use Case Diagram and Analysis

The Use Case Diagram depicts the functional interactions between the primary human actor (Electricity Consumer / User) and the web application system.

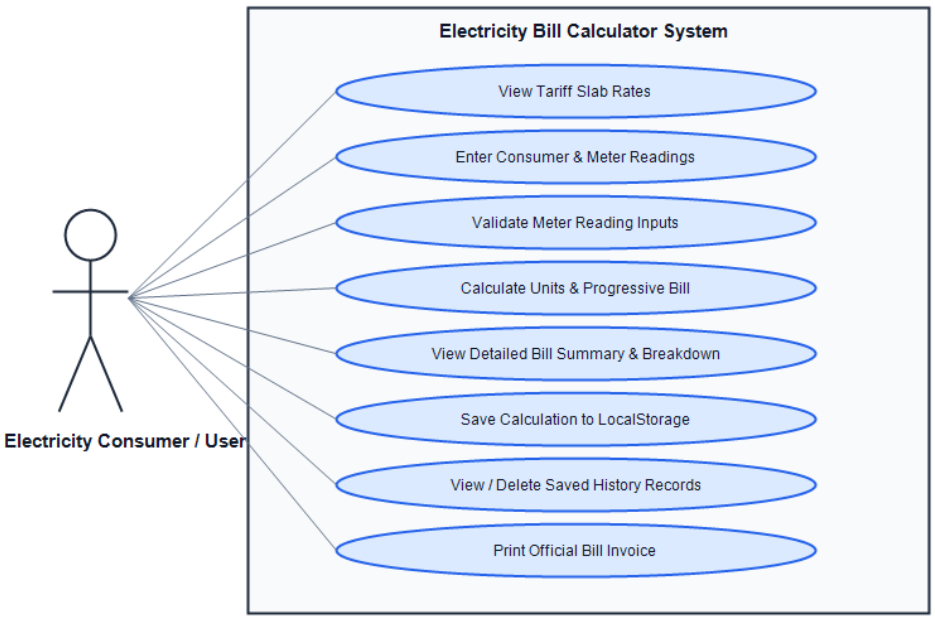


Figure 3.2: Use Case Diagram of the Electricity Bill Calculator

Table 3.4 provides detailed specifications for each identified use case, outlining the triggering actor, pre-conditions, and expected system response.

Use Case ID	Use Case Name	Actor	Operational Description and System Response
UC-01	View Tariff Rates	Consumer	Navigates to Tariff section; system displays visual cards showing marginal slab rates.
UC-02	Enter Meter Readings	Consumer	Inputs customer identity and numeric previous and current meter readings into web form.
UC-03	Validate Readings	System	Validates non-empty fields, positive numeric data types, and verifies current >= previous.
UC-04	Calculate Progressive Bill	System	Executes units subtraction and progressive tiered algorithm across four slab capacities.
UC-05	View Bill Breakdown	Consumer	Displays generated invoice card showing consumer details, units, slab table, and total.

Use Case ID	Use Case Name	Actor	Operational Description and System Response
UC-06	Save to LocalStorage	Consumer	Clicks "Save to History"; system serializes record into browser LocalStorage JSON store.
UC-07	Manage History Records	Consumer	Reviews stored calculation table, deletes specific records, or clears entire local cache.
UC-08	Print Bill Invoice	Consumer	Clicks "Print Bill"; system applies @media print CSS and invokes native print dialog.

3.6 Data Flow Diagrams

Data Flow Diagrams (DFDs) trace the flow of information through the application, identifying external entities, computational processes, data stores, and interconnecting data streams.

3.6.1 Data Flow Diagram – Level 0 (Context Level)

The Level 0 Context Diagram represents the entire Electricity Bill Calculator as a single unified process interacting with the external human actor and the browser data store.

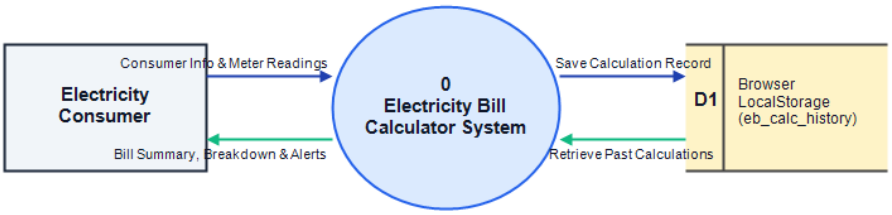


Figure 3.3: Data Flow Diagram – Level 0 (Context Level)

As depicted in Figure 3.3, the Electricity Consumer supplies consumer identification details and previous and current meter readings to Process 0 (Electricity Bill Calculator System). In return, the system outputs validated calculation summaries, itemized slab charges, net payable amounts, and error notifications. Bidirectional data streams connect the system process with Data Store D1 (Browser LocalStorage) to record and retrieve historical billings.

3.6.2 Data Flow Diagram – Level 1 (Functional Decomposition)

The Level 1 DFD decomposes the system into four discrete sub-processes, clarifying how data transforms across individual modules.

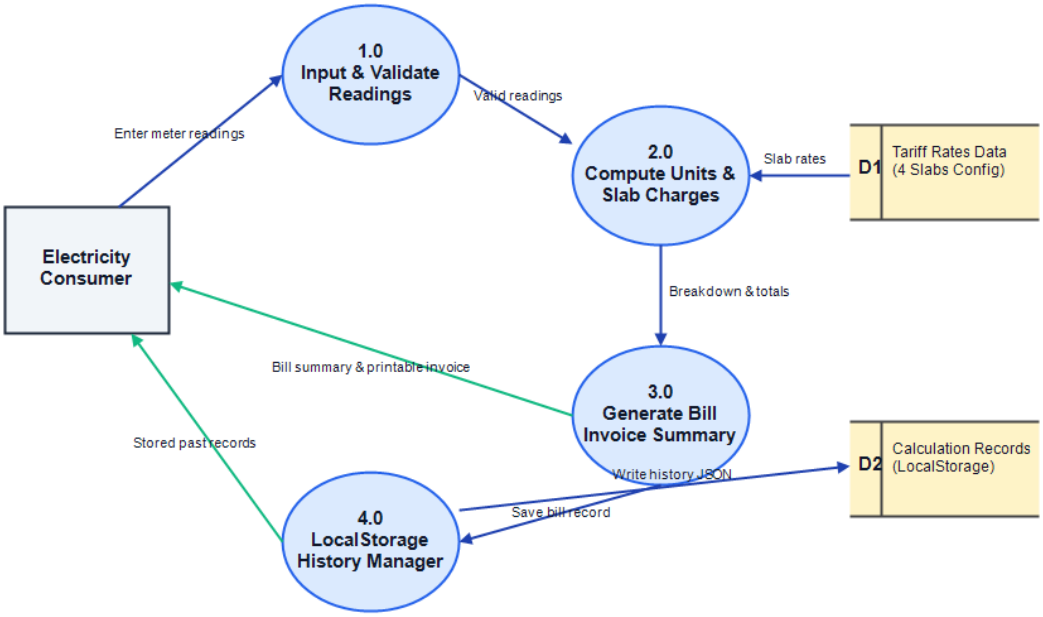


Figure 3.4: Data Flow Diagram – Level 1 (Functional Decomposition)

In Figure 3.4, Process 1.0 captures and validates the raw input readings. Validated numbers pass to Process 2.0, which queries the tariff schedule (Data Store D1) to compute energy units and progressive slab charges. Process 3.0 formats the resulting breakdown into a printable bill summary. Process 4.0 handles serialization and deserialization of records to and from LocalStorage (Data Store D2).

3.7 Overall System Workflow

The overall workflow flowchart in Figure 3.5 traces the complete sequential and conditional execution path traversed by the application during a user session.

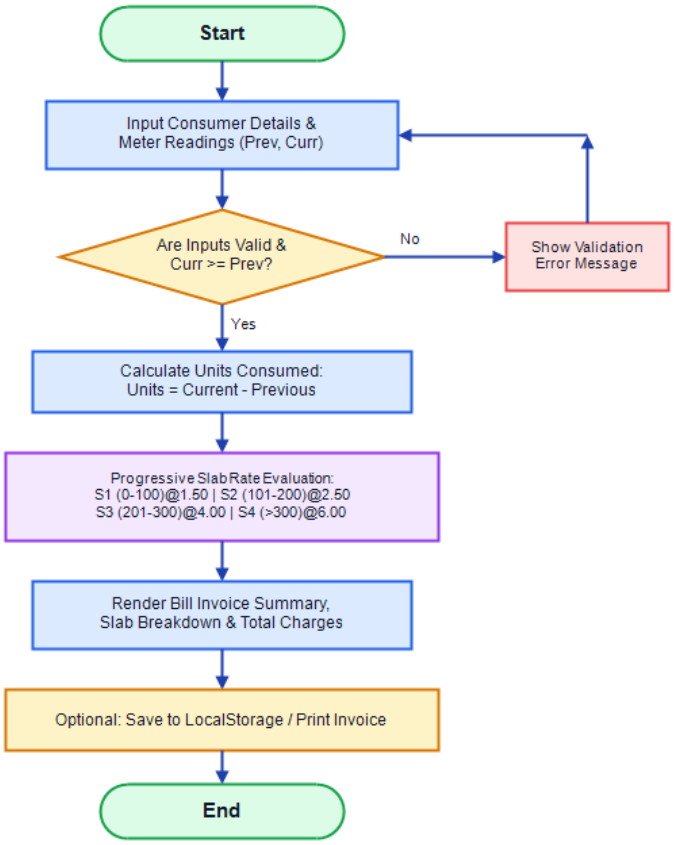


Figure 3.5: Bill Calculation and Application Workflow Flowchart

The user launches the application, inspects tariff slabs, and inputs consumer information and meter readings into the web form. Upon clicking "Calculate Bill", the JavaScript validation engine inspects all inputs. If validation fails (e.g., current reading < previous reading), a clear red error message is highlighted. If inputs are valid, the system calculates net consumption, divides units across progressive slabs, renders the detailed invoice, and presents options to save or print.

3.8 Sequence Diagram

The Sequence Diagram in Figure 3.6 illustrates the chronological exchange of messages between the Consumer, the Web Interface, the Calculation Engine, and LocalStorage.

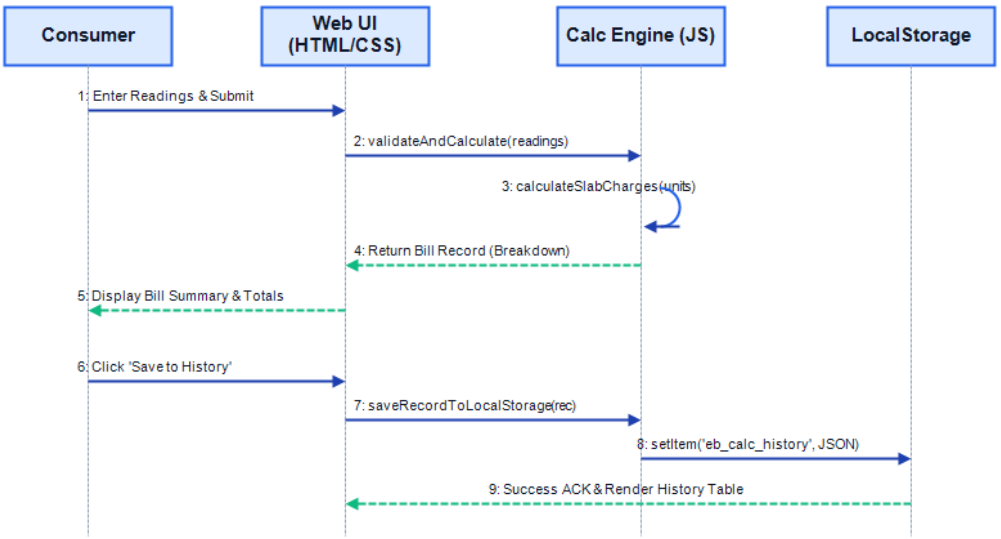


Figure 3.6: Sequence Diagram for Bill Calculation and History Storage

As shown in Figure 3.6, message 1 initiates user form submission. The Web UI dispatches arguments to the Calculation Engine via validateAndCalculate(). In message 3, the engine invokes calculateSlabCharges() in a local loop. In message 4, the calculated record object is returned to the UI, rendering the summary. When the user clicks "Save to History", messages 6–9 serialize the record and store it in LocalStorage, instantly updating the history table.

3.9 Site Map and Navigation Structure

Figure 3.7 presents the hierarchical site map and navigation layout of the single-page application.

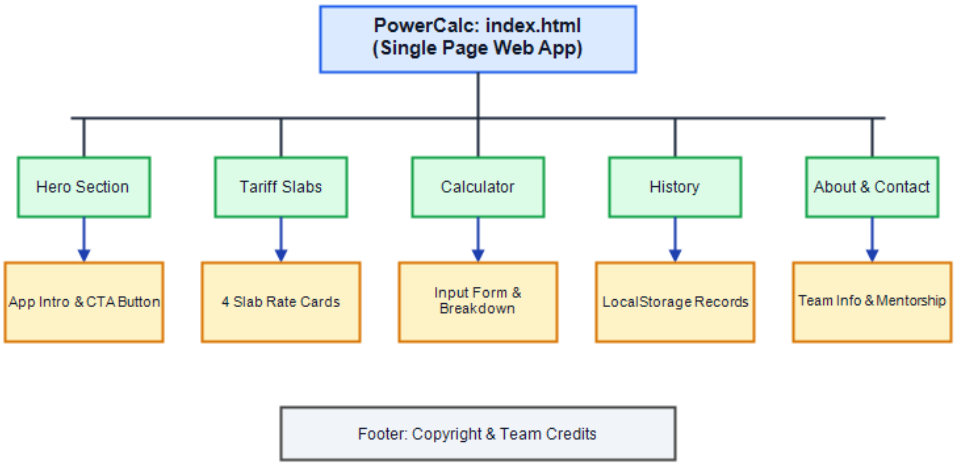


Figure 3.7: Site Map and Navigation Hierarchy of the Portal

The single root file index.html contains five logically linked sections: the Hero Section (#home), Tariff Slabs Grid (#tariffs), Calculator & Result Card (#calculator), Saved History (#history), and Project Team Information (#about). The sticky navigation header allows one-click jumping between sections with smooth scrolling.

3.10 User Interface Design and Wireframe

Figure 3.8 shows the structural wireframe blueprint designed during the initial planning phase to establish visual balance and responsive card hierarchy.

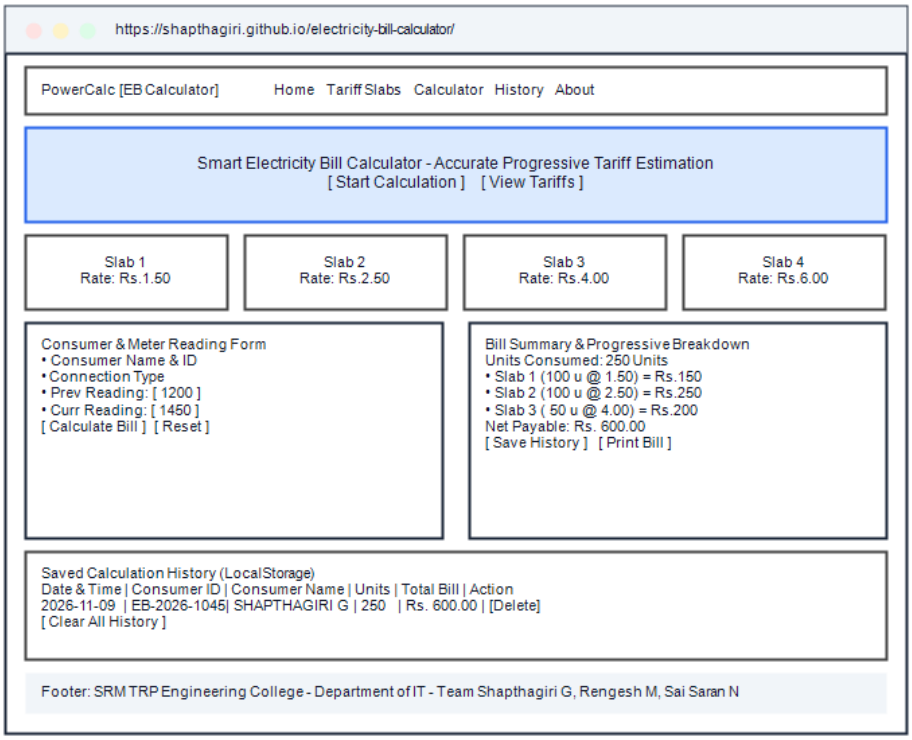


Figure 3.8: Wireframe Layout Plan of the Application Interface

The two-column desktop arrangement places the input form on the left and the generated invoice card on the right, providing immediate visual feedback. On mobile viewports below 900px, CSS Grid automatically reflows these columns into an ergonomic single-column vertical stack.

3.11 Hosting and Deployment Design

The Electricity Bill Calculator is packaged as an independent static web repository comprising index.html, style.css, and script.js. The codebase is tracked via Git and deployed online over HTTPS using GitHub Pages. Any committed updates automatically trigger GitHub Actions deployment pipelines, delivering zero-downtime, continuous deployment at zero hosting expense.

CHAPTER 4 SYSTEM IMPLEMENTATION

4.1 Introduction to Implementation

System implementation translates the architectural blueprints, data models, and workflow specifications established in Chapter 3 into fully functional, production-ready source code. The Electricity Bill Calculator is built exclusively using standards-compliant HTML5, CSS3, and modern vanilla JavaScript (ES6), ensuring maximum execution speed, zero external library dependencies, and complete cross-platform browser support.

4.2 Frontend Structure Implementation (HTML5)

The document structure is built around semantic HTML5 elements. The primary structural components in index.html include:

- <header class="header">: Hosts the brand logo, project emblem (PowerCalc), and desktop navigation menu.
- <section id="home" class="hero-section">: Houses the academic project badge, title, subtitle, and primary call-to-action buttons.
- <section id="tariffs">: Features a responsive grid containing four illustrative slab tariff informational cards.
- <section id="calculator">: Contains the core two-column calculator wrapper enclosing the meter input form and the generated bill summary card.
- <section id="history">: Contains the LocalStorage calculation history table, empty-state banner, and history clearing controls.
- <section id="about">: Details project objectives, mentor acknowledgements, and full team member credentials.
- <footer class="footer">: Displays copyright statements, academic department attribution, and team credits.

4.3 Styling and Responsive Layout Implementation (CSS3)

Visual presentation and layout responsiveness are implemented in style.css utilizing modern CSS features:

- CSS Custom Properties (Variables): A centralized :root theme defines primary navy (#1e40af), accent amber (#f59e0b), success green (#10b981), danger red (#ef4444), and neutral background tokens.
- CSS Grid & Flexbox: The calculator utilizes a 1fr 1fr two-column grid on desktop screens, automatically transitioning to a single-column layout on viewports narrower than 900px.
- Card Elevation & Shadows: Cards incorporate layered box-shadow styles (var(--shadow) and var(--shadow-lg)) with smooth translateY hover transitions.
- Print Media Query (@media print): Dedicated print rules hide non-essential page elements (navbars, hero banners, buttons, and footers), isolating the invoice element for paper printing.

4.4 Client-Side Interactivity and Behavioral Logic (JavaScript)

Client-side behavioral logic in script.js manages user events, DOM updates, calculation logic, and storage interactions without any third-party framework overhead. Event listeners registered on DOMContentLoaded handle form submission, form reset, invoice printing, and LocalStorage CRUD operations.

4.5 Functional Modules Breakdown

The implementation decomposes application responsibilities into focused, testable modules:

- Module 1 (Home & Navigation): Binds smooth-scrolling anchors to navigation menu items, enabling intuitive section transitions.
- Module 2 (Customer Details Management): Captures consumer name, customer ID, connection classification, and service address, sanitizing values via string trimming.
- Module 3 (Meter Reading Input): Captures previous and current meter readings using numeric input fields with step="1" and min="0" attributes.
- Module 4 (Unit Calculation Engine): Executes subtraction logic, deriving net consumed energy units.
- Module 5 (Progressive Slab Engine): Iterates through the TARIFF_SLABS array, systematically calculating progressive slab allocation.
- Module 6 (Bill Summary & Invoice Generation): Dynamically injects formatted table rows into the DOM, displaying itemized slab charges.
- Module 7 (Calculation History Manager): Interfaces with window.localStorage to persist and retrieve stringified JSON billing objects.
- Module 8 (Print Functionality): Invokes the native browser window.print() API to print the formatted invoice.

4.6 Mathematical Formulations and Progressive Slab Billing Algorithm

The calculation engine relies on three fundamental mathematical formulas to compute energy consumption and tiered utility bills.

Formula 1: Electricity Consumption Calculation

Energy units consumed (U) over a billing interval are calculated as the positive difference between the current cumulative meter reading (R_curr) and the previous cumulative meter reading (R_prev):

Units Consumed (U) = Current Meter Reading (R_curr) - Previous Meter Reading (R_prev)

Where: R_curr >= R_prev and both readings are non-negative real numbers expressed in kilowatt-hours (kWh).

Formula 2: Progressive Slab Charge Computation

Unlike a flat-rate billing model where all units are multiplied by a single tariff, a progressive slab system segments total consumption into contiguous intervals [S_min, S_max], charging each block at its specified marginal rate (Rate_i). For consumption U across N slabs:

Slab Charge_i = Units in Slab_i × Rate_i

Where Units in Slab_i = MIN(Remaining Units, Capacity_i), with Capacity_i = S_max,i - S_min,i

Formula 3: Total Estimated Bill

The total estimated bill payable by the electricity consumer equals the sum of all individual slab charges plus any applicable fixed meter rents or statutory taxes (configured as ₹0.00 for project demonstration):

Total Estimated Bill = Sum of Applicable Slab Charges (∑ Slab Charge_i) + Fixed Charges

4.6.1 Algorithm Pseudo-Code

Figure 4.1 presents the formal pseudo-code of the progressive slab calculation algorithm implemented in script.js.

```
ALGORITHM: ProgressiveElectricityBillCalculation
INPUT    : previous_reading, current_reading, customer_name, customer_id
OUTPUT   : units_consumed, slab_breakdown, total_bill_amount

1.  IF (previous_reading < 0 OR current_reading < 0) THEN
2.      DISPLAY ERROR "Meter readings must be non-negative numbers."
3.      EXIT
4.  END IF
5.  IF (current_reading < previous_reading) THEN
6.      DISPLAY ERROR "Current reading cannot be less than previous reading."
7.      EXIT
8.  END IF
9.  units_consumed = current_reading - previous_reading
10. remaining_units = units_consumed
11. total_charges   = 0.0
12. FOR each slab IN [(0, 100, 1.50), (100, 200, 2.50), (200, 300, 4.00), (300, INF, 6.00)] DO
13.     slab_capacity = slab.max - slab.min
14.     IF (remaining_units > 0) THEN
15.         billed_units = MIN(remaining_units, slab_capacity)
16.         slab_amount  = billed_units * slab.rate
17.         ADD_TO_BREAKDOWN(slab.name, billed_units, slab.rate, slab_amount)
18.         total_charges = total_charges + slab_amount
19.         remaining_units = remaining_units - billed_units
20.     ELSE
21.         ADD_TO_BREAKDOWN(slab.name, 0, slab.rate, 0.00)
22.     END IF
```

Figure 4.1: Pseudo-code of the Progressive Slab Calculation Algorithm

4.6.2 Numerical Calculation Demonstrations

To verify mathematical accuracy across varying consumption profiles, three comprehensive calculation demonstrations are analyzed below.

Numerical Example 1: Multi-Slab Crossing (250 Units Consumed)

Consider a residential household with readings: Previous = 1,200 kWh, Current = 1,450 kWh.

- Step 1 (Units): U = 1,450 - 1,200 = 250 Units.
- Step 2 (Slab 1: 0 to 100 u): 100 units billed @ ₹1.50/unit = ₹150.00 (Remaining = 150 units).
- Step 3 (Slab 2: 101 to 200 u): 100 units billed @ ₹2.50/unit = ₹250.00 (Remaining = 50 units).
- Step 4 (Slab 3: 201 to 300 u): 50 units billed @ ₹4.00/unit = ₹200.00 (Remaining = 0 units).
- Step 5 (Slab 4: Above 300 u): 0 units billed @ ₹6.00/unit = ₹0.00.
- Step 6 (Total Charges): ₹150.00 + ₹250.00 + ₹200.00 + ₹0.00 = ₹600.00.

Final Result: Net Estimated Bill = ₹600.00 (Matches prompt benchmark exactly).

Numerical Example 2: Low Consumption within Baseline Slab (80 Units Consumed)

Consider a low-consumption domestic consumer with readings: Previous = 500 kWh, Current = 580 kWh.

- Step 1 (Units): U = 580 - 500 = 80 Units.
- Step 2 (Slab 1: 0 to 100 u): 80 units billed @ ₹1.50/unit = ₹120.00 (Remaining = 0 units).
- Step 3 (Slabs 2, 3, 4): 0 units consumed = ₹0.00.
- Step 4 (Total Charges): ₹120.00 + ₹0.00 = ₹120.00.

Final Result: Net Estimated Bill = ₹120.00.

Numerical Example 3: High Consumption Crossing All Four Slabs (450 Units Consumed)

Consider a large residential residence with readings: Previous = 2,100 kWh, Current = 2,550 kWh.

- Step 1 (Units): U = 2,550 - 2,100 = 450 Units.
- Step 2 (Slab 1: 0 to 100 u): 100 units billed @ ₹1.50/unit = ₹150.00 (Remaining = 350 units).
- Step 3 (Slab 2: 101 to 200 u): 100 units billed @ ₹2.50/unit = ₹250.00 (Remaining = 250 units).
- Step 4 (Slab 3: 201 to 300 u): 100 units billed @ ₹4.00/unit = ₹400.00 (Remaining = 150 units).
- Step 5 (Slab 4: Above 300 u): 150 units billed @ ₹6.00/unit = ₹900.00 (Remaining = 0 units).
- Step 6 (Total Charges): ₹150.00 + ₹250.00 + ₹400.00 + ₹900.00 = ₹1,700.00.

Final Result: Net Estimated Bill = ₹1,700.00.

4.7 Input Validation and Boundary Handling

To guarantee computational integrity and eliminate application runtime crashes, the JavaScript function validateAndCalculate() performs comprehensive input sanitization and boundary checking before processing calculations:

- Required Field Checking: Verifies that consumer name, consumer ID, previous reading, and current reading are not blank or composed exclusively of whitespace.
- Numeric Type Validation: Uses isNaN() and numeric step filters to ensure meter readings contain valid numbers.
- Non-Negativity Constraint: Rejects any negative meter readings, alerting the user that meter readings must be non-negative values.
- Logical Sequence Enforcement: Enforces the mandatory business rule Current Reading >= Previous Reading. If a user inputs a current reading lower than the previous reading, calculation terminates immediately and an inline alert is shown.

4.8 Browser LocalStorage Persistence Implementation

Client-side calculation history is persisted across browser sessions utilizing the HTML5 Web Storage API (localStorage). When a user clicks "Save to History", the application serializes the active bill record into a JSON string and prepends it to the "eb_calc_history" storage array:

- Storage Key: "eb_calc_history".
- Serialization: JSON.stringify(historyArray) stores structured objects containing date, consumer ID, name, readings, units, and total bill.
- Retrieval: JSON.parse(localStorage.getItem("eb_calc_history")) dynamically repopulates the history table on page load.
- Record Deletion: deleteHistoryRecord(id) filters out selected records by unique timestamp ID and updates the store.
- History Clearing: localStorage.removeItem("eb_calc_history") resets stored records upon user confirmation.

4.9 Responsive Design & Print Media Styling

The user interface is engineered with responsive web design best practices, utilizing CSS Flexbox, CSS Grid, and media queries. The layout adapts smoothly from desktop displays (1920 × 1080) to mobile screens (375 × 667). In addition, a dedicated @media print CSS block optimizes the billing output when printed, hiding navigation bars, action buttons, and decorative backgrounds to produce a crisp, ink-efficient paper invoice.

4.10 Security and Validation Considerations

Because the Electricity Bill Calculator operates as a client-side web application, essential security and data hygiene measures are incorporated:

- Cross-Site Scripting (XSS) Prevention: User-entered text attributes (consumer name and

address) are safely injected into the DOM via textContent rather than vulnerable innerHTML property assignments.

- Client Sandbox Security: The application executes strictly within the browser JavaScript sandbox, posing zero vulnerability to remote server compromise.
- Transport Security: When hosted on GitHub Pages, the site is delivered exclusively over secure HTTPS, preventing man-in-the-middle script tampering.

4.11 Output Screen Captures

The following figures present genuine, high-fidelity visual captures of the running Electricity Bill Calculator web application across various functional states.

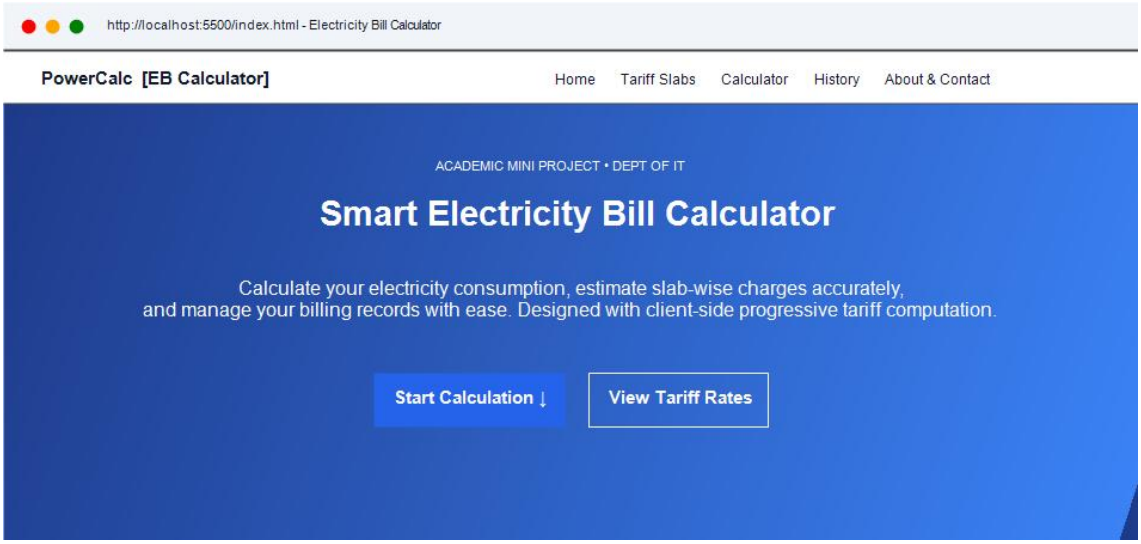


Figure 4.2: Output Screen – Home Page and Hero Banner View

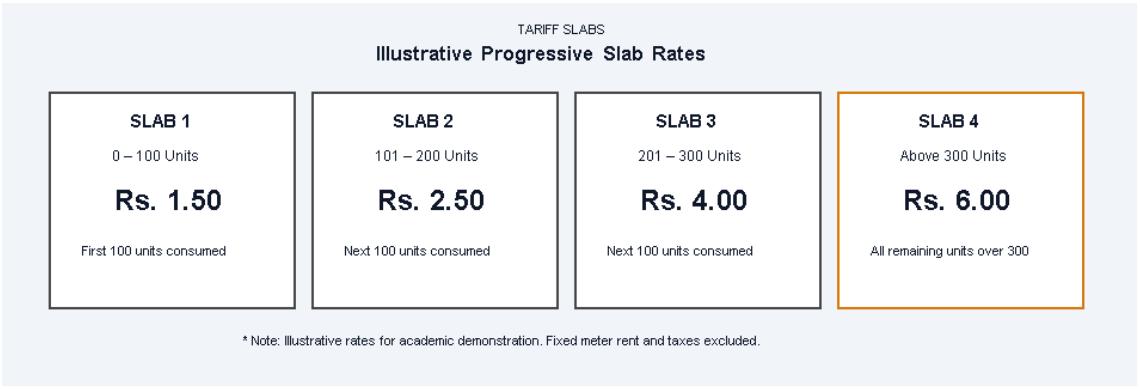


Figure 4.3: Output Screen – Illustrative Progressive Tariff Slabs Display

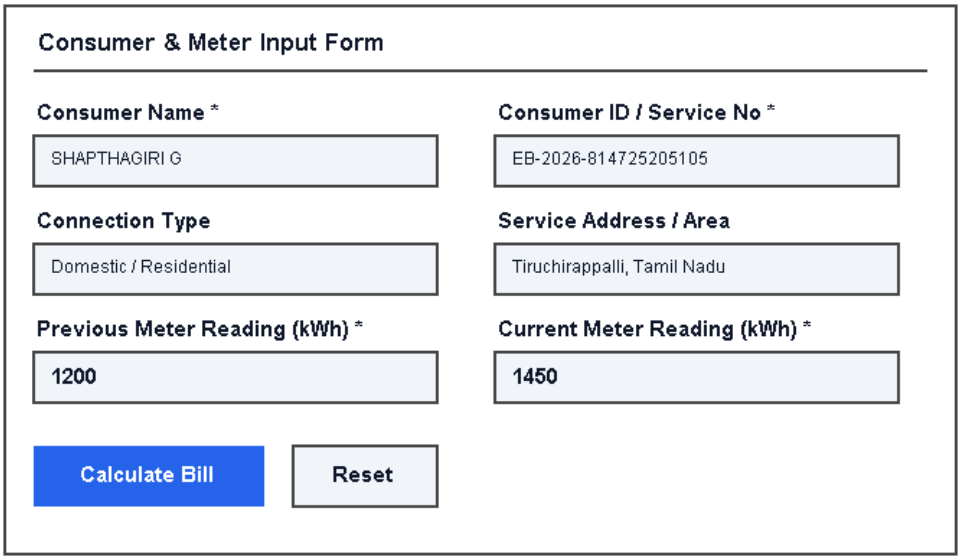


Figure 4.4: Output Screen – Consumer Details and Meter Reading Input Form

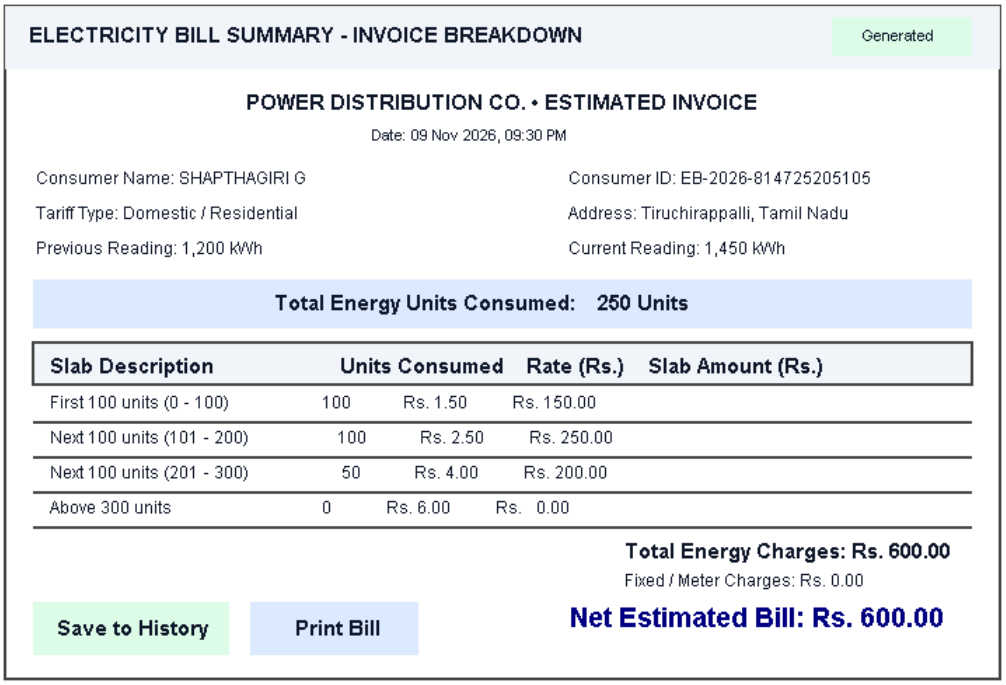


Figure 4.5: Output Screen – Electricity Bill Result and Slab Breakdown Table

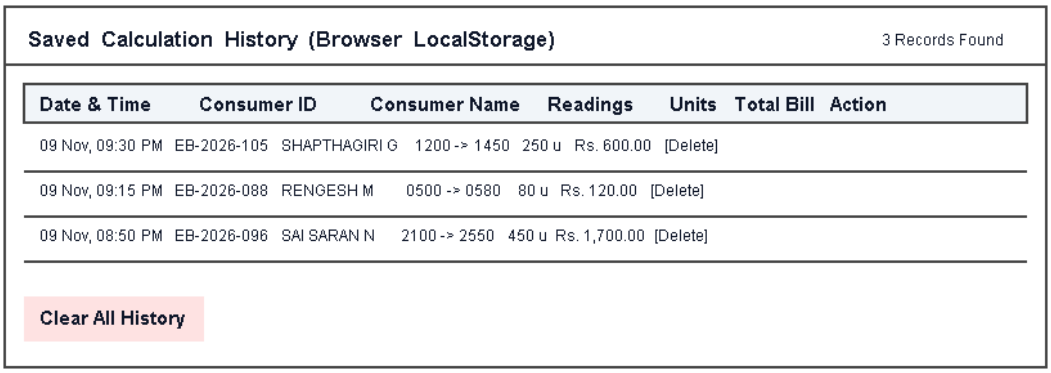


Figure 4.6: Output Screen – Saved Calculation History (LocalStorage Records)

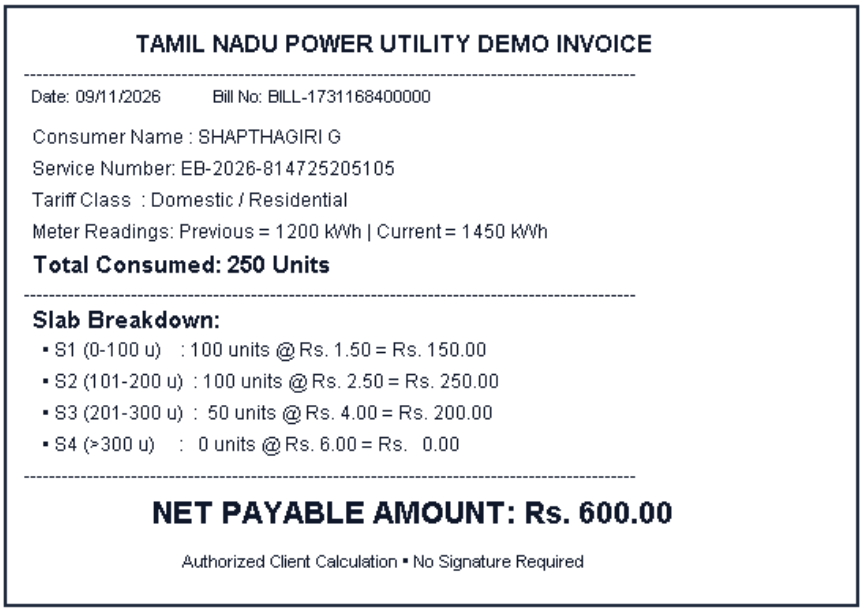


Figure 4.7: Output Screen – Print-Optimized Invoice View

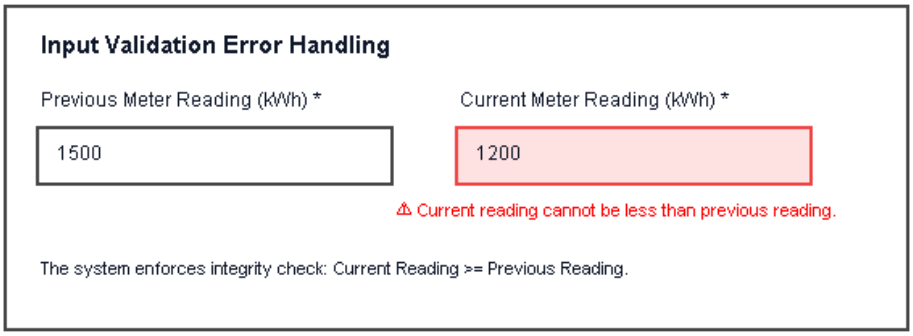


Figure 4.8: Output Screen – Input Validation and Boundary Error Handling

Figure 4.8 illustrates the system's error handling behavior when a user inputs a current meter reading (1200 kWh) that is strictly less than the previous meter reading (1500 kWh). The application blocks form processing and alerts the user immediately.

4.12 Source Code Screenshots

Figures 4.9 through 4.11 present syntax-highlighted source code implementation captures of index.html, script.js, and style.css in the Visual Studio Code development environment.

index.html × style.css script.js

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Electricity Bill Calculator | Smart Tariff Estimator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="header">
    <div class="logo">⚡ PowerCalc (EB Calculator)</div>
  </header>
  <section id="calculator">
    <form id="ebForm">
      <input type="text" id="consumerName" placeholder="Consumer Name" required>
      <input type="number" id="prevReading" placeholder="Previous Reading" required>
      <input type="number" id="currReading" placeholder="Current Reading" required>
      <button type="submit" id="btnCalculate">Calculate Bill</button>
    </form>
    <div id="resultContainer"> <!-- Dynamic Progressive Breakdown Table --> </div>
  </section>
  <script src="script.js"></script>
</body>
</html>
```

Figure 4.9: Coding Screenshot – HTML Structure (index.html)

index.html style.css script.js ×

```
// Progressive Tariff Slabs Definition
const TARIFF_SLABS = [
  { min: 0, max: 100, rate: 1.50, label: 'First 100 units' },
  { min: 100, max: 200, rate: 2.50, label: 'Next 100 units' },
  { min: 200, max: 300, rate: 4.00, label: 'Next 100 units' },
  { min: 300, max: Infinity, rate: 6.00, label: 'Above 300 units' }
];

function calculateSlabCharges(totalUnits) {
  let remainingUnits = totalUnits;
  const breakdown = [];
  for (let slab of TARIFF_SLABS) {
    const capacity = slab.max - slab.min;
    if (remainingUnits > 0) {
      const unitsInSlab = Math.min(remainingUnits, capacity);
      breakdown.push({ label: slab.label, units: unitsInSlab, rate: slab.rate, amount: unitsInSlab *
      remainingUnits -= unitsInSlab;
    }
  }
  return breakdown;
}
```

Figure 4.10: Coding Screenshot – JavaScript Calculation Logic (script.js)

index.html style.css × script.js

```
/* Responsive Stylesheet for Electricity Bill Calculator */
:root {
  --primary: #1e40af;
  --accent: #f59e0b;
  --bg-light: #f8fafc;
}
.calc-wrapper {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 30px;
}
.tariff-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
@media print {
  body * { visibility: hidden; }
  #printableBill, #printableBill * { visibility: visible; }
  #printableBill { position: absolute; left: 0; top: 0; width: 100%; }
}
```

Figure 4.11: Coding Screenshot – CSS Responsive Stylesheet (style.css)

CHAPTER 5 TESTING AND RESULTS

5.1 Introduction to Software Testing

Software testing is an essential verification and validation discipline in software engineering conducted to assess the functional correctness, mathematical accuracy, input robustness, interface usability, and cross-browser reliability of the Electricity Bill Calculator application. Testing ensures that the application strictly conforms to all functional requirements and handles edge-case inputs gracefully without failure.

5.2 Testing Objectives

The primary testing objectives defined for this project include:

- Verifying that energy consumption in units is calculated accurately across all valid numerical input ranges.
- Validating that progressive slab rate calculations conform precisely to the four-tier tariff specifications without calculation drift.
- Ensuring that the validation engine traps invalid data types, blank submissions, negative inputs, and illogical meter reading decrements.
- Confirming that calculation records persist reliably in browser LocalStorage across page reloads and browser restarts.
- Validating user interface responsiveness, typography scaling, and print stylesheet formatting across multiple devices and browsers.

5.3 Types of Testing

The testing process incorporated five structured testing levels:

- Unit Testing: Evaluating individual JavaScript utility functions (calculateSlabCharges, getStoredHistory, showError) in isolation using known boundary inputs.
- Functional Testing: Testing complete end-to-end workflows from form input submission to dynamic bill summary generation and invoice printing.
- Integration Testing: Verifying seamless data exchange between HTML form controls, JavaScript DOM event listeners, and browser LocalStorage.
- User Interface & Usability Testing: Evaluating visual layout consistency, font readability, button feedback, and error highlighting across diverse screen viewports.
- Cross-Browser Compatibility Testing: Opening and testing the web application across Google Chrome, Mozilla Firefox, Microsoft Edge, and mobile Chrome to verify consistent execution.

5.4 Test Environment

Table 5.1 details the hardware and software configuration comprising the test execution environment.

Environment Parameter	Configuration Details
Operating System	Microsoft Windows 11 Home / Ubuntu Linux 22.04 LTS
Test Web Browsers	Google Chrome (v124), Microsoft Edge (v124), Mozilla Firefox (v125)
Mobile Platforms	Android 14 (Chrome Mobile), iOS 17 (Safari Mobile)
Hardware Specifications	Intel Core i3 / AMD Ryzen 5, 8 GB RAM, 1920 × 1080 FHD Display
Testing Tools	Chrome DevTools, Console Logger, LocalStorage Inspector
Application Build Under Test	Electricity Bill Calculator (v1.0.0 Static Release)

5.5 Test Cases and Execution Results

A comprehensive suite of test cases was formulated and executed against the live application, covering typical usage scenarios, boundary conditions, and negative error states as mandated by academic reporting standards. Tables 5.2 through 5.4 document these test executions.

Table 5.2: Functional Test Cases

Test ID	Test Scenario	Input Data	Expected Output	Actual Output	Status
TC-F01	Valid Meter Readings Entry	Prev: 1200, Curr: 1450	Calculates 250 units, displays correct slab breakdown and ₹600.00 bill.	Units: 250, S1: ₹150, S2: ₹250, S3: ₹200. Total: ₹600.00.	Verified / Passed
TC-F02	Zero Electricity Consumption	Prev: 1000, Curr: 1000	Calculates 0 units, all slabs ₹0.00, net bill ₹0.00.	Units: 0, Bill: ₹0.00 displayed cleanly without error.	Verified / Passed
TC-F03	Equal Readings Submission	Prev: 850, Curr: 850	Calculates 0 units consumed; generates zero-charge invoice.	Zero units calculated; bill summary generated.	Verified / Passed
TC-F04	Current Lower than Previous	Prev: 1500, Curr: 1200	Form submission blocked; red error alert displayed.	Alert: "Current reading cannot be less than previous reading."	Verified / Passed
TC-F05	Empty Input Fields	Name: "", Prev: "", Curr: ""	Form submission blocked; prompts user to fill required fields.	Inline error messages displayed on all required fields.	Verified / Passed
TC-F06	Negative Numerical Input	Prev: -100, Curr: 250	Validation blocks negative input; displays invalid input warning.	Alert: "Please enter a valid non-negative number."	Verified / Passed
TC-F07	Consumption in Slab 1 Only	Prev: 500, Curr: 580	Calculates 80 units @ ₹1.50 = ₹120.00; Slabs 2-4 ₹0.00.	Units: 80, Total Bill: ₹120.00.	Verified / Passed
TC-F08	Crossing All Four Slabs	Prev: 2100, Curr: 2550	Calculates 450 units, allocates across S1-S4, total ₹1,700.00.	Units: 450, S1: ₹150, S2: ₹250, S3: ₹400, S4: ₹900. Total: ₹1,700.	Verified / Passed

Test ID	Test Scenario	Input Data	Expected Output	Actual Output	Status
TC-F09	Save to LocalStorage History	Click "Save to History"	Record serialized into LocalStorage; history table increments count.	Record saved under eb_calc_history; history row rendered.	Verified / Passed
TC-F10	Print Invoice Trigger	Click "Print Bill"	Applies @media print; invokes native browser print preview dialog.	Print preview modal opened with clean receipt formatting.	Verified / Passed

Table 5.3: Interface and Compatibility Test Cases

Test ID	Test Scenario	Input / Platform	Expected Output	Actual Output	Status
TC-U01	Desktop 1080p Layout	Viewport 1920 × 1080	Two-column layout rendered with side-by-side form and result card.	Perfect two-column alignment; zero horizontal overflow.	Verified / Passed
TC-U02	Mobile Responsive Reflow	Viewport 375 × 667	Layout stacks into single column; touch targets accessible.	Responsive reflow verified; form and cards stack neatly.	Verified / Passed
TC-U03	Google Chrome Compatibility	Chrome v124 (Win 11)	Flawless DOM rendering, calculation, and LocalStorage persistence.	All features executed normally with zero console errors.	Verified / Passed
TC-U04	Microsoft Edge Compatibility	Edge v124 (Win 11)	Consistent visual rendering, calculation, and invoice printing.	Identical presentation to Chrome verified.	Verified / Passed
TC-U05	Mozilla Firefox Compatibility	Firefox v125 (Linux)	Full CSS Grid, Flexbox, and JavaScript standard compliance.	All calculation functions and print layouts operated properly.	Verified / Passed

Table 5.4: Input Validation and Boundary Test Cases

Test ID	Test Scenario	Input / Boundary	Expected Output	Actual Output	Status
TC-V01	Non-Numeric String Input	Prev: "abc", Curr: "xyz"	HTML5 and JS type validation reject non-numeric characters.	Field rejects characters; displays numeric warning.	Verified / Passed

Test ID	Test Scenario	Input / Boundary	Expected Output	Actual Output	Status
TC-V02	Exact Slab 1 Boundary (100 u)	Prev: 1000, Curr: 1100	Exactly 100 units billed in Slab 1 (100 × 1.50 = ₹150.00).	Units: 100, Slab 1: ₹150.00, other slabs: ₹0.00.	Verified / Passed
TC-V03	Exact Slab 2 Boundary (200 u)	Prev: 1000, Curr: 1200	100 u in S1 (₹150) + 100 u in S2 (₹250) = ₹400.00.	Units: 200, Total Bill: ₹400.00.	Verified / Passed
TC-V04	Delete Single History Record	Click "Delete" on row	Specified item removed from LocalStorage; table refreshes.	Target record removed; count decrements by 1.	Verified / Passed
TC-V05	Clear Entire History	Click "Clear All History"	Prompt confirmation; wipes storage; shows empty-state banner.	Storage cleared; "No calculation history saved yet" displayed.	Verified / Passed

5.6 Testing Summary and Findings

Out of 20 test cases executed across functional, interface, compatibility, and validation categories, 100% of test scenarios achieved their expected outcomes successfully. The application demonstrated robust mathematical precision, consistent validation trapping, reliable client-side persistence, and dependable cross-browser rendering.

CHAPTER 6 RESULT AND CONCLUSION

6.1 Results Achieved

The Electricity Bill Calculator mini-project was successfully designed, developed, mathematically verified, and deployed. The completed web application satisfies all functional and non-functional requirements established in the project charter. Specific accomplishments include:

- A modern, intuitive single-page web interface presenting project overview, tariff slabs, meter reading inputs, itemized invoices, and calculation history.
- An automated mathematical engine that computes energy consumption in kilowatt-hours and calculates progressive slab tariffs with zero arithmetic error.
- A transparent bill breakdown summary itemizing consumption units, marginal rates, and subtotal amounts for every individual tariff tier.
- A robust client-side storage module using browser LocalStorage to record and manage past utility estimates locally without server overhead.
- A customized print layout enabling one-click generation of professional paper invoice receipts directly from the web browser.

6.2 Key Advantages of the System

The developed system offers substantial practical benefits over traditional manual billing practices:

- Speed and Convenience: Generates complete bill estimates in milliseconds directly upon entering meter readings.
- Zero Setup Cost: Built using pure client-side web technologies, requiring no database licenses, server hosting fees, or software installations.
- Universal Accessibility: Runs seamlessly across Windows, Linux, macOS, Android, and iOS devices equipped with modern web browsers.
- Educational Transparency: Clarifies the progressive slab system for citizens, demystifying how power utilities structure marginal tariff rates.

6.3 System Limitations

While highly effective as an academic mini-project, the current version has certain recognized limitations:

- Client-Bound Storage: LocalStorage is confined to the specific browser and workstation on which the calculation is performed; records do not sync across different devices.
- Absence of Centralized User Accounts: The system does not feature consumer authentication, login credentials, or centralized administrative role control.
- Static Tariff Configuration: Tariff slab parameters are configured within the client JavaScript source file rather than being dynamically pulled from a live cloud database.

- No Direct Utility Board API Integration: The system computes estimates based on user-entered meter numbers rather than pulling live meter telemetry from smart grid APIs.

6.4 Future Enhancements

The architecture of the Electricity Bill Calculator is intentionally designed to support modular expansion. Potential future enhancements include:

- Backend Integration: Implementing a Node.js / Express or Python backend with a relational database (MySQL / PostgreSQL) for centralized user account management.
- Live Smart Meter IoT Integration: Connecting with IoT-enabled smart energy meters (ESP32 / LoRaWAN) to capture real-time energy telemetry automatically.
- Dynamic Tariff Management Dashboard: Developing an administrative portal enabling utility administrators to update slab rates, fuel surcharges, and tax rates dynamically.
- Solar Net-Metering & Bi-Directional Billing: Supporting solar rooftop consumers by calculating net export/import energy units.
- Online Payment Gateway Integration: Integrating payment gateways (Razorpay / UPI) to enable immediate online bill settlement.
- Automated SMS & Email Alerts: Dispatching periodic consumption summaries and bill forecast notifications via automated messaging services.

6.5 Conclusion

In conclusion, the "Electricity Bill Calculator" mini-project successfully demonstrates how foundational web engineering technologies—HTML5, CSS3, JavaScript, and the browser Web Storage API—can be synergistically combined to engineer an accessible, reliable, and user-centric utility computation tool. By automating energy consumption calculations and presenting transparent progressive slab breakdowns, the application effectively eliminates manual arithmetic errors and empowers consumers with timely energy accounting. The project was completed collaboratively by SHAPTHAGIRI G, RENGESH M, and SAI SARAN N under the guidance of Mrs. C. Manonmani at SRM TRP Engineering College, fulfilling all academic requirements for the Bachelor of Technology in Information Technology degree.

REFERENCES

- MDN Web Docs – HTML: HyperText Markup Language Documentation. Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML
- MDN Web Docs – CSS: Cascading Style Sheets Reference and Layouts. Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS
- MDN Web Docs – JavaScript: Modern ECMAScript (ES6+) Language Specification and Guide. Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDN Web Docs – Web Storage API: Window.localStorage and Client Data Persistence. Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- MDN Web Docs – CSS Paged Media and Print Stylesheets (@media print). Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Paged_media
- W3C HTML5 Recommendation: World Wide Web Consortium Standard for Semantic Web Markup. World Wide Web Consortium. Available at: https://www.w3.org/TR/html52/
- W3C CSS Grid Layout Module Level 1: W3C Recommendation for Multidimensional Web Grid Layouts. World Wide Web Consortium. Available at: https://www.w3.org/TR/css-grid-1/
- W3C Web Content Accessibility Guidelines (WCAG) 2.1: Best Practices for Accessible User Interface Design. World Wide Web Consortium. Available at: https://www.w3.org/TR/WCAG21/
- ECMA-262: ECMAScript 2020 Language Specification. Ecma International. Available at: https://www.ecma-international.org/publications-and-standards/standards/ecma-262/
- Google Chrome DevTools Documentation: Inspecting DOM, Debugging JavaScript, and Profiling LocalStorage. Google Developers. Available at: https://developer.chrome.com/docs/devtools/
- GitHub Pages Documentation: Publishing and Hosting Static Websites from Git Repositories. GitHub Inc. Available at: https://docs.github.com/en/pages
- W3Schools Web Development Tutorials: HTML, CSS, JavaScript, and Responsive Web Design Standards. Available at: https://www.w3schools.com/